

Exact arithmetic for decisions

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	On ordinary models, the decision is the same	2
4	Where Float64 cannot decide	3
5	Cost, and what carries it	5
6	What “exact” does and does not mean	6
7	Summary	8
8	References	8

1 Overview

Decision variable elimination multiplies probabilities and adds utilities, and both can go wrong in Float64 for the same reason: the quantities that matter can be much smaller than the quantities being carried. A run of observations can drive the evidence mass below `floatmin`, and utilities of opposite sign can cancel, leaving a difference the significand no longer holds.

`DecisionVariableElimination(stable=true)` runs the *same* bucket schedule as the default backend — the same elimination order, the same valuation algebra as the “exhaustive versus DVE” vignette — but interprets the bound data as exact rationals and rounds only the returned value. Exact arithmetic was chosen in preference to taking logarithms, because utilities are signed and have no logarithm, and in preference to a fixed working precision, which can still lose small terms before they cancel.

This vignette is as interested in where the default path is *fine* as in where it is not. The short answer, developed below: on ordinary models the two agree on the decision and differ in the last digit or two of the value. When the evidence underflows, Float64 cannot tell rare evidence from

impossible evidence, so the default backend does not try: it detects the untrustworthy mass and reruns the same schedule in exact arithmetic by itself.

2 Setup

```
using InfluenceDiagrams
using BayesianNetworks
using FiniteKernels
using Random
```

3 On ordinary models, the decision is the same

Take twelve hundred random single-decision diagrams, over utility magnitudes from 1 to 10^{17} , and compare the two paths on both the value and the recovered policy:

```
function random_case(rng, scale)
    id = influence_diagram(:C ⇒ [:c1, :c2, :c3], :D ⇒ [:a1, :a2];
        mechanisms = [:C ⇒ ()], decisions = [:D ⇒ ()],
        utilities = [:U ⇒ (:C, :D)])

    p = rand(rng, 3)
    p ./= sum(p)
    m = bind_cpt(InfluenceDiagramModel(id), :C ⇒ p)
    m = bind_utility(m, :U ⇒ (rand(rng, 3, 2) .- 0.5) .* scale)
    return optimize(m), optimize(m, DecisionVariableElimination(stable = true))
end

function survey()
    rng = MersenneTwister(11)
    values_differ = 0
    policies_differ = 0
    worst = 0.0
    total = 0
    for scale in (1e0, 1e8, 1e16, 1e17), _ in 1:300
        float_sol, exact_sol = random_case(rng, scale)
        total += 1
        float_sol.expected_utility == exact_sol.expected_utility || (values_differ
+= 1)
        worst = max(worst,
            abs(float_sol.expected_utility - exact_sol.expected_utility) /
            max(abs(exact_sol.expected_utility), 1e-300))
        policy_table(float_sol.strategy[:D]) == policy_table(exact_sol.strategy[:D])
        ||
            (policies_differ += 1)
    end
    return (cases = total, values_differ = values_differ,
```

```

        policies_differ = policies_differ, worst_relative = worst)
end
survey()

```

```

(cases = 1200, values_differ = 584, policies_differ = 0, worst_relative =
1.617525970693216e-14)

```

Roughly half the cases differ in the returned value, and the largest relative difference is a few units in the last place. Not one differs in the *policy*. On this evidence a user who wants the recommended action, and not the sixteenth digit of its value, has no reason to leave the default path.

4 Where Float64 cannot decide

The interesting failure is not inaccuracy, it is a false conclusion. Consider a decision taken after a long survey in which a rare species is detected at every site:

```

function survey_model(n)
    sites = [Symbol("D", i) for i in 1:n]
    id = influence_diagram([s ⇒ [:no, :yes] for s in sites]...,
        :Act ⇒ [:protect, :ignore];
        mechanisms = vcat([sites[1] ⇒ ()],
            [sites[i] ⇒ (sites[i - 1],) for i in
2:n]),
        decisions = [:Act ⇒ ()],
        utilities = [:U ⇒ (sites[n], :Act)])
    m = bind_cpt(InfluenceDiagramModel(id), sites[1] ⇒ [0.999, 0.001])
    for i in 2:n
        m = bind_cpt(m, sites[i] ⇒ [0.999 0.001; 0.9 0.1])
    end
    return bind_utility(m, :U ⇒ [0.0 10.0; 100.0 0.0]), sites
end

m, sites = survey_model(340)
observed = observe(m, [s ⇒ :yes for s in sites[1:(end - 1)]])

```

```
InfluenceDiagramModel(341 variables, 1 decision, 1 utility, 340 kernels, 1 bound,
evidence on D1, D10, D100, D101, D102, D103, D104, D105, D106, D107, D108, D109,
D11, D110, D111, D112, D113, D114, D115, D116, D117, D118, D119, D12, D120, D121,
D122, D123, D124, D125, D126, D127, D128, D129, D13, D130, D131, D132, D133, D134,
D135, D136, D137, D138, D139, D14, D140, D141, D142, D143, D144, D145, D146, D147,
D148, D149, D15, D150, D151, D152, D153, D154, D155, D156, D157, D158, D159, D16,
D160, D161, D162, D163, D164, D165, D166, D167, D168, D169, D17, D170, D171, D172,
D173, D174, D175, D176, D177, D178, D179, D18, D180, D181, D182, D183, D184, D185,
D186, D187, D188, D189, D19, D190, D191, D192, D193, D194, D195, D196, D197, D198,
D199, D2, D20, D200, D201, D202, D203, D204, D205, D206, D207, D208, D209, D21,
D210, D211, D212, D213, D214, D215, D216, D217, D218, D219, D22, D220, D221, D222,
D223, D224, D225, D226, D227, D228, D229, D23, D230, D231, D232, D233, D234, D235,
D236, D237, D238, D239, D24, D240, D241, D242, D243, D244, D245, D246, D247, D248,
D249, D25, D250, D251, D252, D253, D254, D255, D256, D257, D258, D259, D26, D260,
D261, D262, D263, D264, D265, D266, D267, D268, D269, D27, D270, D271, D272, D273,
D274, D275, D276, D277, D278, D279, D28, D280, D281, D282, D283, D284, D285, D286,
D287, D288, D289, D29, D290, D291, D292, D293, D294, D295, D296, D297, D298, D299,
D3, D30, D300, D301, D302, D303, D304, D305, D306, D307, D308, D309, D31, D310,
D311, D312, D313, D314, D315, D316, D317, D318, D319, D32, D320, D321, D322, D323,
D324, D325, D326, D327, D328, D329, D33, D330, D331, D332, D333, D334, D335, D336,
D337, D338, D339, D34, D35, D36, D37, D38, D39, D4, D40, D41, D42, D43, D44, D45,
D46, D47, D48, D49, D5, D50, D51, D52, D53, D54, D55, D56, D57, D58, D59, D6, D60,
D61, D62, D63, D64, D65, D66, D67, D68, D69, D7, D70, D71, D72, D73, D74, D75, D76,
D77, D78, D79, D8, D80, D81, D82, D83, D84, D85, D86, D87, D88, D89, D9, D90, D91,
D92, D93, D94, D95, D96, D97, D98, D99)
```

That evidence is unlikely but entirely possible. Its probability is about 10^{-340} , below the smallest positive Float64, so the Float64 run of the bucket algorithm ends with an evidence mass of zero. Zero mass is also what a contradiction produces. Rather than draw a conclusion Float64 cannot support, the default backend treats any mass that is not a normal positive number as undecided, and reruns the schedule in exact rational arithmetic (ADR 0014):

```
sol = optimize(observed)
sol.expected_utility, policy_table(sol.strategy[:Act]),
sol.diagnostics.exact_fallback
```

```
(10.0, fill(:protect), true)
```

The chain is Markov, so this is checkable by hand: conditioning on the previous site leaves the last one with posterior $[0.9, 0.1]$, and the utility table gives $0.9 \times 0 + 0.1 \times 100 = 10$ for `:protect` against $0.9 \times 10 + 0.1 \times 0 = 9$ for `:ignore`.

`exact_fallback` records that the rerun happened, and the answer is the one the explicit exact path gives:

```
exact = optimize(observed, DecisionVariableElimination(stable = true))
exact.expected_utility == sol.expected_utility
```

```
true
```

The diagnostics say what happened rather than leaving it to be inferred:

```
d = sol.diagnostics
(mass_status = d.mass_status, evidence_probability = d.evidence_probability,
 log_evidence_probability = round(d.log_evidence_probability; digits = 1),
 arithmetic = d.arithmetic, exact_probability_guards = d.exact_probability_guards)
```

```
(mass_status = :underflow, evidence_probability = 0.0, log_evidence_probability =
-785.2, arithmetic = :exact_rational, exact_probability_guards = true)
```

evidence_probability is 0.0 because that is the nearest Float64, while log_evidence_probability is finite and mass_status is :underflow rather than :zero. ImpossibleEvidenceError is kept for evidence whose probability is exactly zero, which only the exact rerun can establish:

```
contradiction = bind_cpt(observed, sites[1] ⇒ [1.0, 0.0])
try
    optimize(contradiction)
catch e
    typeof(e)
end
```

ImpossibleEvidenceError

This is the decision-side counterpart of the log-domain layer in BayesianNetworkInference.jl: the same distinction between “too small to represent” and “cannot happen”, reached through exact rationals rather than logarithms, because utilities are signed.

5 Cost, and what carries it

Exact rational arithmetic is more expensive per factor cell than Float64. What it does not do is change the *shape* of the computation: the bucket algorithm still scales with factor width rather than with the number of policies. A decision observing twelve binary signals has 2^{12} deterministic policies:

```
function wide_information(k)
    signals = [Symbol("S", i) for i in 1:k]
    id = influence_diagram([s ⇒ [:lo, :hi] for s in signals]..., :Act ⇒ [:go, :stop];
                           mechanisms = [s ⇒ () for s in signals],
                           decisions = [:Act ⇒ Tuple(signals)],
                           utilities = [:U ⇒ (signals[1], :Act)])
    m = InfluenceDiagramModel(id)
    for s in signals
        m = bind_cpt(m, s ⇒ [0.5, 0.5])
    end
    return bind_utility(m, :U ⇒ [0.0 5.0; 8.0 0.0])
end
```

```

wide = wide_information(12)
wide_sol = optimize(wide, DecisionVariableElimination(stable = true))
(exhaustive = try
  optimize(wide, ExhaustivePolicySearch()).expected_utility
catch e
  typeof(e)
end,
exact_dve = wide_sol.expected_utility,
max_factor_size = wide_sol.diagnostics.max_factor_size)

```

```
(exhaustive = PolicySearchTooLargeError, exact_dve = 6.5, max_factor_size = 4)
```

The oracle refuses; the bucket algorithm answers from a largest factor of four cells. Only S1 reaches the utility, so the optimal policy is “go when S1 is high, stop when it is low”, worth $0.5 \times 8 + 0.5 \times 5 = 6.5$. To be clear about the attribution: that scaling belongs to decision variable elimination, not to exact arithmetic — the default backend solves this case too. The point is that choosing exact arithmetic does not cost it.

6 What “exact” does and does not mean

It means the arithmetic on the bound data is exact. It does not mean the model is exact, and the difference shows up immediately:

```

two_stage = two_stage_model()
(float_path = optimize(two_stage).expected_utility,
 exact_path = optimize(two_stage,
  DecisionVariableElimination(stable = true)).expected_utility)

```

```
(float_path = 21.0, exact_path = 21.000000000000004)
```

The exact path returns the *uglier* number, and that is not a defect. A CPT entry written 0.1 is bound as the Float64 nearest to a tenth, which is not a tenth; the exact rational value of the model as actually bound is the longer number, and the default path’s clean 21.0 is a rounding that happens to land on the round figure. Exact arithmetic answers “what does this bound model imply?”, not “what did you mean to type?”.

Two further limits, both deliberate. Rounded CPTs are not silently renormalised: a model whose rows sum to one only within the tolerance is accepted, and `exact_probability_guards` then distinguishes exact constancy from mere tolerance acceptance, so a run reports which it had. And `expected_utility(m, strategy; stable=true)` evaluates a *given* strategy in the same arithmetic, so a strategy found on one path can be scored on the other:

```

two_stage_strategy = optimize(two_stage,
  DecisionVariableElimination(stable = true)).strategy
(exact = expected_utility(two_stage, two_stage_strategy; stable = true),
 float = expected_utility(two_stage, two_stage_strategy))

```

```
(exact = 21.000000000000007, float = 21.0)
```

The exact figure here is not bit-identical to the one optimize returned above. Both are correctly rounded, but the two routes group the rational operations differently – the bucket algorithm eliminates variable by variable, the evaluator sums over the whole joint – and correct rounding of two different exact expressions need not land on the same Float64. “Exact” constrains the arithmetic along a route; it does not make two routes agree in the last bit.

Tolerated rounding has a limit of its own. A model may hold entries a hair below zero, within the tolerance it is bound at, because rounded tables produce them. When the evidence is so improbable that its mass is no larger than what those entries could shift it by, the sign of the answer is an artefact of the rounding, and no answer is returned:

```
tiny = influence_diagram(:X ⇒ [:a, :b], :Y ⇒ [:a, :b], :Act ⇒ [:leave, :act];
                        mechanisms = [:X ⇒ (), :Y ⇒ (:X,)],
                        decisions = [:Act ⇒ ()], utilities = [:V ⇒ (:Y, :Act)])
rounded = bind_cpt(InfluenceDiagramModel(tiny),
                  [:X ⇒ [1 - 1e-6, 1e-6], :Y ⇒ [0.5 0.5; 1 + 1e-7 -1e-7]];
                  atol = 1e-6)
rounded = observe(bind_utility(rounded, :V ⇒ [0.0 1.0; 2.0 3.0]), :X ⇒ :b)
try
  optimize(rounded; atol = 1e-6)
catch e
  typeof(e)
end
```

IndeterminatePosteriorError

The evidence mass here is 10^{-6} . That is inside the tolerance budget $(1 + 10^{-6})^3 - 1 \approx 3 \times 10^{-6}$, which counts three mechanisms: the two chance mechanisms and the decision’s policy, as in the network a strategy instantiates. Removing the negative entry, or binding at a tolerance that rejects it, resolves the error.

The negative entry counts because it lies on a state the evidence allows, $X = b$ and $Y = b$. In the row of $X = a$, which the evidence rules out, it takes no part in the posterior, and the answer is determined. The exhaustive search gives the same answer:

```
ruled_out = bind_cpt(InfluenceDiagramModel(tiny),
                    [:X ⇒ [1 - 1e-6, 1e-6], :Y ⇒ [1 + 1e-7 -1e-7; 0.5 0.5]];
                    atol = 1e-6)
ruled_out = observe(bind_utility(ruled_out, :V ⇒ [0.0 1.0; 2.0 3.0]), :X ⇒ :b)
optimize(ruled_out; atol = 1e-6).expected_utility,
optimize(ruled_out, ExhaustivePolicySearch(); atol = 1e-6).expected_utility
```

```
(2.0, 2.0)
```

Note also that scoring a strategy goes through the brute-force joint evaluator rather than the bucket algorithm, so unlike optimize it is bounded by the number of joint states: the 340-site model above has 2^{340} of them and cannot be scored this way at any precision.

7 Summary

`stable=true` changes the arithmetic, not the algorithm. On ordinary models it agrees with the default path on every decision and differs in the last place or two of the value, which is a good reason to leave the default alone. When the evidence mass underflows, `Float64` cannot tell possible evidence from impossible evidence; the default path recognises this and reruns in exact arithmetic by itself, so it returns the right decision with `exact_fallback` set, together with a finite log mass and a `mass_status` saying why the ordinary number was zero. `ImpossibleEvidenceError` means probability exactly zero, and a model whose tolerated negative entries leave the answer's sign undetermined raises `IndeterminatePosteriorError` instead of guessing. Signed utilities are what rule out the logarithmic remedy used for posteriors, and rounded-decimal inputs are why “exact” is a statement about the arithmetic rather than about the model.

8 References